# SmartCollect Phoenix - Cognitive Architecture

Camada cognitiva que da ao agente capacidade de raciocinar antes de agir, manter modelo mental do cliente, selecionar estrategias por contexto, e refletir sobre resultados. Aplicada seletivamente em nodes de decisao para balancear inteligencia e custo.

## 1. Problema

### 1.1 Agente Reativo

O SmartCollect atual (V1 e Phoenix base) opera num loop reativo:

```
input → classify intent → call handler → respond
```

O agente nao raciocina sobre a situacao, nao lembra o que ja tentou, nao avalia se a estrategia esta funcionando. Cada turno e tratado isoladamente.

### 1.2 Estrategias por Regra Fixa

A selecao de estrategia de negociacao e um `if/elif` baseado em desconto e perfil:

| Condicao | Estrategia |
|---|---|
| Desconto >= 40% | `highlight_large_discount` |
| Desconto >= 25% | `emphasize_savings` |
| Cliente frustrado | `DEFUSE` / `EMPATHIZE` |

Nao ha adaptacao baseada em: o que ja tentou, o que funcionou com clientes similares, ou qual o nivel de confianca do agente na abordagem.

### 1.3 O que Falta

| Capacidade | Status Atual | Impacto |
|---|---|---|
| Raciocinio antes de agir | Ausente | Respostas genericas, sem estrategia |
| Modelo mental do cliente | Ausente | Agente "esquece" objecoes e tentativas |

| Capacidade | Status Atual | Impacto |
|---|---|---|
| Reflexao pos-acao | Ausente | Nao aprende dentro da conversa |
| Metacognicao | Ausente | Repete abordagens falhas, nao escala quando perdido |
| Selecao de estrategia contextual | Regra fixa | Ignora historico e perfil |

# 2. Cognitive Loop

## 2.1 Arquitetura

O loop cognitivo substitui o padrao reativo em nodes de decisao:

```
┌──────────────────────────────────────────────────────────┐
│                    COGNITIVE NODE                        │
│                                                          │
│  ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌───────┐│
│  │  REASON  │──►│  DECIDE  │──►│ EXECUTE  │──►│REFLECT│
│  │          │   │          │   │          │   │       │
│  │ Analisa  │   │ Escolhe  │   │ Gera     │   │Avalia │
│  │ contexto,│   │estrategia│   │ resposta │   │result,│
│  │ historico│   │com base  │   │ com LLM  │   │update │
│  │ emocao,  │   │no reason │   │ forte    │   │mental │
│  │ tentativas│  │          │   │          │   │model  │
│  └──────────┘   └──────────┘   └──────────┘   └───────┘│
│       │                                          │      │
│       │          ┌──────────────┐                │      │
│       └─────────►│ Mental Model │◄───────────────┘      │
│                  │  (Store API) │                       │
│                  └──────────────┘                       │
└──────────────────────────────────────────────────────────┘
```

## 2.2 Fases

| Fase | LLM Call | Modelo | Latencia | O que faz |
|---|---|---|---|---|
| **Reason** | Sim | GPT-4o-mini | ~500ms | Analisa situacao, historico, emocao, tentativas. Produz `Reasoning` estruturado |

| Fase | LLM Call | Modelo | Latencia | O que faz |
|---|---|---|---|---|
| **Decide** | Nao | Logica | ~10ms | Seleciona estrategia com base no Reasoning + Mental Model + Policy (ver `17-REINFORCEMENT-LEARNING.md` ) |
| **Execute** | Sim | GPT-4o | ~800ms | Gera resposta usando estrategia selecionada e contexto completo |
| **Reflect** | Sim | GPT-4o-mini | ~400ms | Avalia resultado, atualiza Mental Model, seta confidence |

Total overhead: ~1.7s por turno cognitivo.

## 2.3 Aplicacao Seletiva

O Cognitive Loop NAO e aplicado em todos os nodes. Apenas em nodes de decisao marcados com `cognitive: true` no YAML do flow (ver `15-FLOW-COMPOSITION-ENGINE.md` ):

| Node Type | Cognitive | Justificativa |
|---|---|---|
| `onboarding` | Nao | Mecanico (coleta CPF/nome) |
| `transaction` | Nao | Mecanico (MCP call) |
| `negotiation_pitch` | **Sim** | Decisao de como apresentar oferta |
| `persuasion` | **Sim** | Decisao de estrategia de convencimento |
| `option_selection` | Nao | Mecanico (lista e seleciona) |
| `renegotiation` | Nao | Mecanico (match por valor) |
| `confirmation` | Nao | Mecanico (apresenta resumo) |
| `terms_acceptance` | Nao | Mecanico (apresenta termos) |
| `agreement` | Nao | Mecanico (MCP call) |
| `closing` | Nao | Mecanico (despedida) |

Estimativa: ~30-40% dos turnos passam pelo Cognitive Loop.

## 2.4 Implementacao

```python
async def cognitive_node(state: FlowState, config: dict) -> Command:
    ctx: SmartCollectContext = config["configurable"]["context"]
    mental_model = await load_mental_model(ctx.store, state)

    reasoning = await reason(state, mental_model, config)

    strategy = decide_strategy(reasoning, mental_model, config)

    result = await execute_with_strategy(state, strategy, config)

    reflection = await reflect(state, reasoning, strategy, result, config)

    updated_model = update_mental_model(mental_model, reflection)
    await save_mental_model(ctx.store, state, updated_model)

    return Command(
        update={
            "messages": [result.response],
            "cognitive_trace": CognitiveTrace(
                reasoning=reasoning,
                strategy=strategy,
                reflection=reflection,
            ).to_dict(),
            "mental_model": updated_model.to_dict(),
            "last_condition": reflection.next_condition,
        },
        goto="__resolve__",
    )
```

# 3. ClientMentalModel

## 3.1 Estrutura

```python
from dataclasses import dataclass, field
from decimal import Decimal


@dataclass(frozen=True)
class FinancialProfile:
    estimated_capacity: Decimal | None
    preferred_payment_method: str | None
    price_sensitivity: float
    mentioned_constraints: list[str]


@dataclass(frozen=True)
class Objection:
    type: str
    text: str
    turn: int
    resolved: bool


@dataclass(frozen=True)
class StrategyAttempt:
    strategy: str
    turn: int
    outcome: str
    client_reaction: str


@dataclass(frozen=True)
class EmotionSnapshot:
    emotion: str
    intensity: float
    turn: int


@dataclass(frozen=True)
class ClientMentalModel:
    financial_profile: FinancialProfile
    emotional_trajectory: list[EmotionSnapshot]
    objections_raised: list[Objection]
    strategies_attempted: list[StrategyAttempt]
    engagement_level: float
```

```python
    trust_level: float
    negotiation_style: str
    turn_count: int
    key_facts: list[str]

    def with_updates(self, **kwargs) -> "ClientMentalModel":
        from dataclasses import asdict
        current = asdict(self)
        current.update(kwargs)
        return ClientMentalModel(**current)
```

## 3.2 Campos

| Campo | Tipo | Descricao | Fonte |
|---|---|---|---|
| `financial_profile` | `FinancialProfile` | Capacidade financeira estimada, sensibilidade a preco, restricoes mencionadas | Inferido das mensagens |
| `emotional_trajectory` | `list[EmotionSnapshot]` | Sequencia de emocoes ao longo da conversa | `EmotionalSupervisor` (ver `11-EMOTIONAL-INTELLIGENCE.md` ) |
| `objections_raised` | `list[Objection]` | Objecoes do cliente (valor alto, nao reconhece, sem dinheiro) com status de resolucao | Classificadas pelo Reason |
| `strategies_attempted` | `list[StrategyAttempt]` | Estrategias usadas e seus resultados | Registradas pelo Reflect |
| `engagement_level` | float (0-1) | Nivel de engajamento do cliente (respostas | Inferido das mensagens |

| Campo | Tipo | Descricao | Fonte |
|---|---|---|---|
| | | longas, perguntas, emojis) | |
| `trust_level` | float (0-1) | Nivel de confianca do cliente no agente | Inferido de objecoes resolvidas, tom cooperativo |
| `negotiation_style` | str | Estilo do cliente: `analytical`, `emotional`, `pragmatic`, `resistant` | Classificado pelo Reason |
| `turn_count` | int | Numero de turnos na conversa | Contador |
| `key_facts` | list[str] | Fatos importantes mencionados ("tenho filhos", "perdi emprego", "recebo dia 15") | Extraidos pelo Reason |

## 3.3 Persistencia

Armazenado no Store API com namespace `("cognitive", thread_id)`:

```python
async def load_mental_model(store: BaseStore, state: FlowState) -> ClientMentalModel:
    thread_id = state["conversation_id"]
    cached = await store.aget(("cognitive", thread_id), "mental_model")

    if cached and cached.value:
        return ClientMentalModel(**cached.value)

    return ClientMentalModel(
        financial_profile=FinancialProfile(
            estimated_capacity=None,
            preferred_payment_method=None,
            price_sensitivity=0.5,
            mentioned_constraints=[],
        ),
        emotional_trajectory=[],
        objections_raised=[],
        strategies_attempted=[],
        engagement_level=0.5,
        trust_level=0.5,
        negotiation_style="unknown",
        turn_count=0,
        key_facts=[],
    )


async def save_mental_model(
    store: BaseStore, state: FlowState, model: ClientMentalModel
) -> None:
    thread_id = state["conversation_id"]
    from dataclasses import asdict
    await store.aput(("cognitive", thread_id), "mental_model", asdict(model))
```

O Mental Model persiste entre turnos da mesma conversa. Em conversas novas do mesmo CPF, o agente pode carregar dados historicos do namespace `("clients", cpf)` via `BusinessContextMemory` .

# 4. Strategy Reasoning

## 4.1 Fase Reason

Prompt estruturado que analisa a situacao e produz um `Reasoning` :

```
REASON_PROMPT = """Analise a situacao atual da negociacao.

<client_mental_model>
{mental_model_json}
</client_mental_model>

<current_message>
{user_message}
</current_message>

<conversation_stage>
{current_stage}
</conversation_stage>

<debt_context>
{debt_summary}
</debt_context>

Responda SOMENTE com JSON valido:
{{
    "situation_analysis": "string (2-3 frases)",
    "client_intent": "string (o que o cliente quer agora)",
    "emotional_state": "string (emocao dominante e intensidade)",
    "objections_active": ["lista de objecoes nao resolvidas"],
    "key_facts_new": ["novos fatos mencionados nesta mensagem"],
    "engagement_delta": float (-1 a 1, mudanca no engajamento),
    "trust_delta": float (-1 a 1, mudanca na confianca),
    "negotiation_style_update": "string ou null",
    "financial_signals": ["sinais sobre capacidade financeira"]
}}"""


@dataclass(frozen=True)
class Reasoning:
    situation_analysis: str
    client_intent: str
    emotional_state: str
    objections_active: list[str]
    key_facts_new: list[str]
    engagement_delta: float
    trust_delta: float
    negotiation_style_update: str | None
    financial_signals: list[str]
```

## 4.2 Fase Decide

Selecao de estrategia sem LLM call, baseada no Reasoning + Mental Model + Strategy Policy:

```python
def decide_strategy(
    reasoning: Reasoning,
    mental_model: ClientMentalModel,
    config: dict,
) -> StrategyDecision:
    policy = config["configurable"].get("strategy_policy")
    available = get_available_strategies(config)

    already_tried = {s.strategy for s in mental_model.strategies_attempted}
    failed = {
        s.strategy for s in mental_model.strategies_attempted
        if s.outcome == "rejected"
    }

    candidates = [s for s in available if s.id not in failed]

    if not candidates:
        return StrategyDecision(
            strategy="empathy_last_resort",
            confidence=0.3,
            fallback="escalate_to_human",
        )

    if policy:
        segment = classify_segment(mental_model)
        weights = policy.get_weights(segment)
        scored = [
            (s, weights.get(s.id, 0.1) * (0.7 if s.id in already_tried else 1.0))
            for s in candidates
        ]
        scored.sort(key=lambda x: x[1], reverse=True)
        best = scored[0]
        return StrategyDecision(
            strategy=best[0].id,
            confidence=best[1],
            fallback=scored[1][0].id if len(scored) > 1 else "escalate_to_human",
        )

    return _rule_based_fallback(reasoning, mental_model, candidates)
```

A `strategy_policy` vem do Reinforcement Learning (ver `17-REINFORCEMENT-LEARNING.md` secao 8). Se nao estiver disponivel, usa `_rule_based_fallback` com regras simples.

## 4.3 Fase Reflect

Avalia o resultado e atualiza o Mental Model:

```
REFLECT_PROMPT = """Avalie o resultado da interacao.

<strategy_used>
{strategy_name}: {strategy_description}
</strategy_used>

<agent_response>
{response_text}
</agent_response>

<client_mental_model_before>
{mental_model_before}
</client_mental_model_before>

Responda SOMENTE com JSON valido:
{{
    "strategy_effectiveness": float (0-1),
    "client_reaction_type": "string (positive|neutral|negative|exit)",
    "next_condition": "string (interested|not_interested|exit|renegotiation|...)",
    "confidence": float (0-1),
    "mental_model_updates": {{
        "engagement_delta": float,
        "trust_delta": float,
        "new_objections": ["lista"],
        "resolved_objections": ["lista"],
        "key_facts": ["lista"]
    }},
    "learning": "string (1 frase sobre o que aprender desta interacao)"
}}"""
```

A Reflect phase e executada APOS o `interrupt()` retornar com a resposta do cliente, nao apos gerar a resposta do agente. Isso permite avaliar como o cliente reagiu a estrategia.

# 5. Metacognicao

## 5.1 CognitiveAssessment

O agente avalia sua propria confianca e identifica lacunas:

```python
@dataclass(frozen=True)
class CognitiveAssessment:
    confidence: float
    uncertainty_areas: list[str]
    information_gaps: list[str]
    recommended_action: str


def metacognitive_check(
    reasoning: Reasoning,
    mental_model: ClientMentalModel,
) -> CognitiveAssessment:
    failed_count = sum(
        1 for s in mental_model.strategies_attempted
        if s.outcome == "rejected"
    )

    if failed_count >= 3:
        return CognitiveAssessment(
            confidence=0.2,
            uncertainty_areas=["exhausted_known_strategies"],
            information_gaps=[],
            recommended_action="escalate_to_human",
        )

    if mental_model.trust_level < 0.2:
        return CognitiveAssessment(
            confidence=0.3,
            uncertainty_areas=["very_low_trust"],
            information_gaps=["real_objection_unknown"],
            recommended_action="ask_clarifying_question",
        )

    if not mental_model.financial_profile.estimated_capacity:
        return CognitiveAssessment(
            confidence=0.5,
            uncertainty_areas=["payment_capacity_unknown"],
            information_gaps=["monthly_income", "other_debts"],
            recommended_action="probe_financial_capacity",
        )

    if reasoning.engagement_delta < -0.3:
        return CognitiveAssessment(
            confidence=0.4,
            uncertainty_areas=["losing_engagement"],
            information_gaps=[],
            recommended_action="change_approach",
```

```
        )

    return CognitiveAssessment(
        confidence=min(mental_model.trust_level + 0.2, 1.0),
        uncertainty_areas=[],
        information_gaps=[],
        recommended_action="proceed",
    )
```

## 5.2 Acoes da Metacognicao

| `recommended_action` | O que o Agente Faz |
|---|---|
| `proceed` | Segue com a estrategia decidida |
| `ask_clarifying_question` | Faz pergunta aberta para entender melhor a situacao |
| `probe_financial_capacity` | Pergunta gentilmente sobre capacidade de pagamento |
| `change_approach` | Muda para estrategia oposta a que vinha usando |
| `escalate_to_human` | Encerra com `Command(goto="human_fallback")` |

## 5.3 Integracao com Cognitive Loop

```python
async def cognitive_node(state: FlowState, config: dict) -> Command:
    ctx: SmartCollectContext = config["configurable"]["context"]
    mental_model = await load_mental_model(ctx.store, state)

    reasoning = await reason(state, mental_model, config)

    assessment = metacognitive_check(reasoning, mental_model)

    if assessment.recommended_action == "escalate_to_human":
        return Command(
            update={
                "last_condition": "human_request",
                "cognitive_trace": CognitiveTrace(
                    reasoning=reasoning,
                    strategy=None,
                    reflection=None,
                    metacognition=assessment,
                ).to_dict(),
            },
            goto="__resolve__",
        )

    strategy = decide_strategy(
        reasoning, mental_model, config,
        override_action=assessment.recommended_action,
    )

    result = await execute_with_strategy(state, strategy, config)

    reflection = await reflect(state, reasoning, strategy, result, config)

    updated_model = update_mental_model(mental_model, reflection)
    await save_mental_model(ctx.store, state, updated_model)

    return Command(
        update={
            "messages": [result.response],
            "cognitive_trace": CognitiveTrace(
                reasoning=reasoning,
                strategy=strategy,
                reflection=reflection,
                metacognition=assessment,
            ).to_dict(),
            "mental_model": updated_model.to_dict(),
            "last_condition": reflection.next_condition,
```

```
        },
        goto="__resolve__",
    )
```

# 6. Cognitive Trace

## 6.1 Estrutura

```
@dataclass(frozen=True)
class CognitiveTrace:
    reasoning: Reasoning | None
    strategy: StrategyDecision | None
    reflection: Reflection | None
    metacognition: CognitiveAssessment | None

    def to_dict(self) -> dict:
        from dataclasses import asdict
        return {
            "reasoning": asdict(self.reasoning) if self.reasoning else None,
            "strategy": asdict(self.strategy) if self.strategy else None,
            "reflection": asdict(self.reflection) if self.reflection else None,
            "metacognition": asdict(self.metacognition) if self.metacognition else None,
        }
```

## 6.2 Uso

O `CognitiveTrace` e persistido no state a cada turno cognitivo e serve para:

| Consumidor | Como Usa |
|---|---|
| **Reinforcement Learning** | `CognitiveTrace.strategy` + `CognitiveTrace.reflection` alimentam `NegotiationExperience` (ver `17-REINFORCEMENT-LEARNING.md`) |
| **Analytics** | Dashboard de estrategias usadas, confidence medio, taxa de escalacao |
| **Auditoria** | Rastreio do raciocinio do agente por conversa |
| **DynamicPromptBuilder** | Injeta `<cognitive_context>` com reasoning resumido para o proximo turno |

## 6.3 Integracao com DynamicPromptBuilder

O `DynamicPromptBuilder` (ver `08-CONTEXT-ENGINEERING.md`) ganha uma nova secao condicional:

```python
def _build_cognitive_context(self, state: FlowState) -> str:
    trace = state.get("cognitive_trace")
    if not trace or not trace.get("reasoning"):
        return ""

    model = state.get("mental_model", {})
    return f"""<cognitive_context>
Analise anterior: {trace['reasoning'].get('situation_analysis', '')}
Estrategia usada: {trace.get('strategy', {}).get('strategy', 'nenhuma')}
Confianca: {trace.get('metacognition', {}).get('confidence', 'N/A')}
Trust level: {model.get('trust_level', 'N/A')}
Objecoes ativas: {', '.join(trace['reasoning'].get('objections_active', []))}
</cognitive_context>"""
```

# 7. Performance e Latencia

## 7.1 Overhead por Turno

| Componente | Sem Cognitive | Com Cognitive | Delta |
|---|---|---|---|
| Intent classification | ~300ms | ~300ms | 0 |
| Reason (GPT-4o-mini) | - | ~500ms | +500ms |
| Decide (logica local) | - | ~10ms | +10ms |
| Execute (GPT-4o) | ~800ms | ~800ms | 0 |
| Reflect (GPT-4o-mini) | - | ~400ms | +400ms |
| Mental Model I/O | - | ~50ms | +50ms |
| **Total** | **~1.1s** | **~2.1s** | **+~1.0s** |

## 7.2 Impacto em Volume (5k conversas/dia)

| Metrica | Sem Cognitive | Com Cognitive Seletivo |
|---|---|---|
| LLM calls/dia | ~150k | ~220k |
| Calls cognitivas/dia | 0 | ~70k (Reason + Reflect) |
| % turnos com cognitive | 0% | ~30-40% |
| Latencia media (p50) | ~1.1s | ~1.4s (media ponderada) |

| Metrica | Sem Cognitive | Com Cognitive Seletivo |
|---|---|---|
| Latencia p95 cognitivo | - | ~2.5s |

## 7.3 Otimizacao de Custo

| Fase | Modelo | Custo por 1k tokens (input/output) | Calls/dia | Custo/dia |
|---|---|---|---|---|
| Reason | GPT-4o-mini | $0.15/0.60$ | ~35k | ~$15 |
| Execute | GPT-4o | $2.50/10.00$ | ~150k | ~$250 |
| Reflect | GPT-4o-mini | $0.15/0.60$ | ~35k | ~$15 |
| **Total** | | | **~220k** | **~$280/dia** |
| **Sem Cognitive** | GPT-4o | $2.50/10.00$ | ~150k | **~$250/dia** |

Overhead de custo: ~$30/dia (~12%) pelo Reason + Reflect em modelo leve.

## 7.4 Alternativa com Modelo Local

Para reduzir custo e latencia do Reason/Reflect:

| Fase | Modelo | Latencia | Custo |
|---|---|---|---|
| Reason | Qwen3-4B (vLLM local) | ~200ms | Infra only |
| Reflect | Qwen3-4B (vLLM local) | ~150ms | Infra only |

Com modelo local para Reason/Reflect, overhead de latencia cai para ~400ms e custo cognitivo vai para ~$0/dia (apenas custo de infra GPU/CPU).

# 8. Integracao com EmotionalSupervisor

A `emotional_trajectory` no `ClientMentalModel` e alimentada pelo `EmotionalSupervisor` (ver `11-EMOTIONAL-INTELLIGENCE.md`):

```python
def update_mental_model(
    model: ClientMentalModel,
    reflection: Reflection,
    emotional_analysis: dict | None = None,
) -> ClientMentalModel:
    new_trajectory = list(model.emotional_trajectory)

    if emotional_analysis and emotional_analysis.get("sentiment"):
        sentiment = emotional_analysis["sentiment"]
        new_trajectory.append(EmotionSnapshot(
            emotion=sentiment.primary_emotion.value,
            intensity=sentiment.intensity,
            turn=model.turn_count + 1,
        ))

    new_strategies = list(model.strategies_attempted)
    if reflection.strategy:
        new_strategies.append(StrategyAttempt(
            strategy=reflection.strategy,
            turn=model.turn_count + 1,
            outcome=reflection.client_reaction_type,
            client_reaction=reflection.learning,
        ))

    new_engagement = max(0.0, min(1.0,
        model.engagement_level + reflection.mental_model_updates.get("engagement_delta", 0)
    ))
    new_trust = max(0.0, min(1.0,
        model.trust_level + reflection.mental_model_updates.get("trust_delta", 0)
    ))

    return model.with_updates(
        emotional_trajectory=new_trajectory,
        strategies_attempted=new_strategies,
        engagement_level=new_engagement,
        trust_level=new_trust,
        turn_count=model.turn_count + 1,
        key_facts=list(set(model.key_facts + reflection.mental_model_updates.get("key_facts", []
        objections_raised=_merge_objections(
            model.objections_raised,
            reflection.mental_model_updates.get("new_objections", []),
            reflection.mental_model_updates.get("resolved_objections", []),
            model.turn_count + 1,
        ),
    )
```

O `EmotionalSupervisor` roda ANTES do Cognitive Loop (na middleware stack ou no inicio do node). O resultado emocional alimenta tanto o `DynamicPromptBuilder` (como ja faz no Phoenix base) quanto o `ClientMentalModel`.

# 9. Estrategias Disponiveis

Catalogo de estrategias que o Decide pode selecionar:

| ID | Nome | Descricao | Melhor para |
|---|---|---|---|
| `cash_highlight` | Destaque do desconto a vista | Enfatiza economia total com pagamento unico | Clientes com capacidade, cooperativos |
| `installment_flexibility` | Flexibilidade de parcelas | Mostra opcoes de parcelamento acessiveis | Clientes com restricao financeira |
| `empathy_first` | Empatia antes de oferta | Valida sentimento, depois apresenta solucao | Clientes frustrados ou ansiosos |
| `benefit_immediate` | Beneficio imediato | Destaca limpar nome, recuperar credito | Clientes pragmaticos |
| `urgency_scarcity` | Urgencia e escassez | Condicao especial por tempo limitado | Clientes indecisos, engajados |
| `social_proof` | Prova social | Muitos clientes ja resolveram nessas condicoes | Clientes resistentes |
| `reframe_value` | Reenquadramento de valor | Transforma valor total em valor diario/semanal | Clientes sensiveis a preco |
| `last_resort_empathy` | Empatia como ultimo recurso | Reconhece dificuldade, oferece o melhor possivel | Apos multiplas rejeicoes |
| `clarifying_question` | Pergunta de esclarecimento | Busca entender objecao real | Quando metacognicao detecta incerteza |

As estrategias sao templates de instrucao injetados no prompt do Execute. Novas estrategias podem ser adicionadas sem mudar codigo, apenas adicionando ao catalogo.

# 10. Diferenca do Phoenix Base

| Aspecto | Phoenix Base | Com Cognitive Architecture |
|---|---|---|
| Raciocinio | Ausente | Reason phase com LLM leve |
| Modelo mental | Ausente | `ClientMentalModel` persistido no Store |
| Selecao de estrategia | Regra fixa (if/elif) | Contextual com policy + reasoning |
| Reflexao | Ausente | Reflect phase avalia resultado |
| Metacognicao | Ausente | `CognitiveAssessment` com escalacao inteligente |
| Overhead | 0 | ~1s por turno cognitivo (~30-40% dos turnos) |
| Custo adicional | 0 | ~$30/diacomGPT-4o-mini$, 0 com modelo local |
| Auditabilidade | Logs de resposta | `CognitiveTrace` completo por turno |
| Aprendizado intra-conversa | Nao | Mental Model atualizado a cada turno |